ChatGPT

Algorithm Summaries and Examples

Linear Search

[1] Linear Search scans through a list one element at a time, checking each element against the target value until it's found or the list ends. It works on **any list** (sorted or unsorted) and is straightforward: loop through the array and compare. It's most useful for **small or unsorted lists** where simplicity is needed. For example, checking each entry in a guest list for a particular name uses linear search.

- **Time Complexity:** Worst-case $O(n)$, Best-case $O(1)$; **Space:** $O(1)$.
- **When/Why Used:** When data is unsorted or small. It's easy to implement and requires no extra memory, but is slow for large lists. Example: finding a number in a short list of phone numbers by checking each in turn.

```
def linear_search(arr, target):
    for i, num in enumerate(arr):
        if num == target:
            return i
    return -1

# Example usage
numbers = [4, 2, 7, 1, 3]
print(linear_search(numbers, 7))  # Output: 2 (index of 7)
print(linear_search(numbers, 5))  # Output: -1 (not found)
```

Explanation: The function loops through `arr`. When `num == target`, it returns the index; if it finishes the loop, it returns -1.
Analogy: *"Searching for a name by checking each page one by one."*

Binary Search

[2] [3] Binary Search finds a target in a **sorted** array by repeatedly dividing the search interval in half. You compare the target to the middle element: if equal, you're done; if the target is smaller, continue on the left half; otherwise, on the right half. This **divide-and-conquer** approach quickly narrows down the location.

- **Time Complexity:** $O(\log n)$; **Space:** $O(1)$.
- **When/Why Used:** For searching in a sorted list or array. It dramatically outperforms linear search on large, sorted data. Example: looking up a word in a dictionary (you flip to middle, determine which half to continue).

```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

# Example usage (array must be sorted)
nums = [1, 3, 5, 7, 9]
print(binary_search(nums, 7))   # Output: 3
print(binary_search(nums, 2))   # Output: -1
```

Explanation: The code maintains `left` and `right` pointers. It repeatedly checks `arr[mid]` and moves the pointers inward depending on comparison with `target`.
Analogy: *"Looking for a word in a dictionary by skipping ahead to the midpoint and narrowing down."*

Merge Sort

4 Merge Sort is a divide-and-conquer sorting algorithm that **recursively** splits the array in half, sorts each half, and then merges the sorted halves back together. Because it splits the data evenly, then merges back, it guarantees a good worst-case performance.

- **Time Complexity:** $O(n \log n)$ for best, average, and worst cases; **Space:** $O(n)$ auxiliary.
- **When/Why Used:** When you need a stable, reliable sort and have enough memory for the extra array. It's great for large datasets and external sorting (sorting data that doesn't fit in memory) because of its predictable runtime. Example: merging two sorted lists of names into one sorted list.

```python
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    merged = []
    i = j = 0
    # Merge two sorted halves
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            merged.append(left[i]); i += 1
        else:
```

```
            merged.append(right[j]); j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged

# Example usage
print(merge_sort([5, 2, 9, 1, 5, 6]))  # Output: [1,2,5,5,6,9]
```

Explanation: The function splits `arr` into halves, recursively sorts them, and then merges by always taking the smaller head element of the two halves.
Analogy: *"Sort by splitting the deck of cards in half, sorting each half, then merging them as you would when riffling two sorted piles."*

Quick Sort

[5] [6] Quick Sort is another divide-and-conquer sorting algorithm. It selects a **pivot** element (often the middle or random), then partitions the array into elements less than the pivot and greater than the pivot, and recursively sorts the partitions. In place versions swap elements so it uses little extra memory.

- **Time Complexity:** Average $O(n \log n)$, worst-case $O(n^2)$ (e.g. bad pivot choices); **Space:** $O(\log n)$ average (for recursion stack).
- **When/Why Used:** Generally faster in practice due to low constant factors and cache efficiency, if we pick pivots well (e.g. random). Common in libraries (like `sort()` in many languages) when you can risk worst-case rarely. Example: sorting an array of numbers quickly (often used in competitive programming).

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + mid + quick_sort(right)

# Example usage
print(quick_sort([3,6,8,10,1,2,1]))  # Output: [1,1,2,3,6,8,10]
```

Explanation: This code picks a `pivot`, splits `arr` into three lists (`left`, `mid`, `right`), recursively sorts `left` and `right`, then concatenates.
Analogy: *"Picking a pivot card from the deck and partitioning cards into those smaller and bigger than it, then recursively sorting each pile."*

Heap Sort

[7] [8] Heap Sort is a comparison-based sorting algorithm that builds a **binary heap** (often a max-heap) from the array, then repeatedly extracts the maximum element and rebuilds the heap, resulting in a sorted list. It's essentially an optimized version of selection sort using a heap data structure.

- **Time Complexity:** $O(n \log n)$ worst-case (and average); **Space:** $O(1)$ auxiliary (in-place heap implementation).
- **When/Why Used:** When you need guaranteed $O(n \log n)$ performance and low extra space. It's often used when memory is tight and worst-case speed matters. Example: priority queues use the heap concept; heap sort uses that to sort a list.

```
import heapq
def heap_sort(arr):
    h = []
    for value in arr:
        heapq.heappush(h, value)
    return [heapq.heappop(h) for _ in range(len(h))]

# Example usage
print(heap_sort([3,2,1,5,6,4]))  # Output: [1,2,3,4,5,6]
```

Explanation: This code uses Python's `heapq` to create a min-heap. It pushes all values onto the heap, then pops them (smallest first), yielding a sorted list. In practice, an in-place heap can sort without extra list. **Analogy:** *"Think of building a pyramid of cards (heap) and then repeatedly removing the top card to make a sorted line."*

Activity Selection

[9] The Activity Selection problem is a **greedy algorithm** for scheduling. Given a set of activities with start and finish times, it selects the maximum number of non-overlapping activities. The trick is to always pick the next activity that finishes earliest (after sorting by finish time), then continue with the next activity that starts after that finish time.

- **Time Complexity:** $O(n \log n)$ (dominated by sorting by finish times); **Space:** $O(n)$.
- **When/Why Used:** Useful for scheduling tasks, booking rooms, or any scenario of maximizing usage without overlap. Example: Scheduling meetings in a single room to accommodate as many as possible by always choosing the meeting that ends soonest.

```
def activity_selection(activities):
    activities.sort(key=lambda x: x[1])  # sort by finish time
    selected = []
    last_finish = 0
    for start, finish in activities:
```

```
        if start >= last_finish:
            selected.append((start, finish))
            last_finish = finish
    return selected

# Example usage
acts = [(1,4), (3,5), (0,6), (5,7), (3,9), (5,9), (6,10), (8,11)]
print(activity_selection(acts))  # Example result: [(1,4), (5,7), (8,11)]
```

Explanation: After sorting by finish time, we pick each activity whose start time is after the `last_finish`. This yields a maximum set of non-overlapping activities.
Analogy: *"Choosing which talks to attend by always picking the one that ends soonest, so you can fit more talks in."*

Huffman Coding

10 Huffman Coding is an optimal **prefix-free encoding** algorithm used in lossless data compression. It builds a binary tree of symbols by frequency: more frequent symbols get shorter codes. The algorithm repeatedly merges the two lowest-frequency nodes into a combined node, building up a tree. The resulting codes (paths in the tree) ensure no code is a prefix of another (prefix-free), achieving minimal average code length for given frequencies.

- **Time Complexity:** Building the Huffman tree takes $O(n \log n)$ (using a priority queue), where n is the number of distinct symbols; **Space:** $O(n)$.
- **When/Why Used:** Widely used in compression (e.g. in ZIP, JPEG, MP3). You use it when you want efficient encoding where more common items get fewer bits. Example: encoding letters in a message where letter 'e' appears often and gets a short bit pattern, while rare letters get longer patterns.

```
import heapq
from collections import Counter, namedtuple

class Node(namedtuple("Node", ["freq", "char", "left", "right"])):
    def __lt__(self, other):
        return self.freq < other.freq

def huffman_codes(frequencies):
    heap = [Node(freq, char, None, None) for char, freq in frequencies.items()]
    heapq.heapify(heap)
    while len(heap) > 1:
        node1 = heapq.heappop(heap)
        node2 = heapq.heappop(heap)
        merged = Node(node1.freq + node2.freq, None, node1, node2)
        heapq.heappush(heap, merged)
    root = heap[0]
    codes = {}
```

```
    def generate_codes(node, prefix=""):
        if node.char is not None:
            codes[node.char] = prefix
        else:
            generate_codes(node.left, prefix + "0")
            generate_codes(node.right, prefix + "1")
    generate_codes(root)
    return codes

# Example usage
freqs = Counter("huffman coding example")
codes = huffman_codes(freqs)
print(codes)  # Outputs variable-length binary codes for each character
```

Explanation: We build a min-heap of frequency nodes, merge two smallest each time, and generate codes by traversing the tree (left = 0, right = 1). Each character ends up with a binary code.
Analogy: *"Like giving shorter names (codes) to more common items so you can communicate faster; for example, calling frequently used items by nicknames."*

Kruskal's (MST)

[11] [12] Kruskal's Algorithm finds a **Minimum Spanning Tree (MST)** of a weighted undirected graph by greedily adding edges. It starts with all vertices as separate components, sorts all edges by weight, and then iterates through the edges, adding an edge if it connects two previously unconnected components (no cycle). This uses a Union-Find (disjoint set) data structure to check connectivity.

- **Time Complexity:** $O(E \log E)$ or $O(E \log V)$ for sorting edges; **Space:** $O(V + E)$.
- **When/Why Used:** For finding the cheapest way to connect all nodes (e.g. network design, road systems) where the graph is sparse or edges can be easily sorted. Example: Building roads connecting all cities with minimal total length by always choosing the shortest available road that doesn't form a loop.

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]
    def union(self, x, y):
        self.parent[self.find(x)] = self.find(y)

def kruskal(edges, num_nodes):
    uf = UnionFind(num_nodes)
    mst = []
    for w, u, v in sorted(edges):
```

```
        if uf.find(u) != uf.find(v):
            uf.union(u, v)
            mst.append((u, v, w))
    return mst

# Example usage
edges = [(7,0,1), (5,0,2), (8,1,2), (9,1,3), (7,2,3),
         (5,2,4), (15,3,4), (6,3,5), (8,4,5), (9,4,6), (11,5,6)]
mst = kruskal(edges, 7)
print(mst)  # Example MST edges (vertex u, v, weight)
```

Explanation: Sort edges by weight. For each edge `(u,v)`, if `u` and `v` are in different components (checked via `find`), we `union` them and include the edge. This builds the MST.
Analogy: *"Like connecting islands by bridges: always build the cheapest bridge that doesn't create an already-existing loop."*

Prim's (MST)

13 Prim's Algorithm also finds an MST by growing a single tree. It starts from an arbitrary node and repeatedly adds the cheapest edge that connects the growing tree to a new vertex. This is done using a priority queue (min-heap) of edges crossing the cut between the tree and remaining vertices.

- **Time Complexity:** $O(E + V \log V)$ (with a binary heap) or $O(E \log V)$; **Space:** $O(V + E)$.
- **When/Why Used:** Like Kruskal's, for network design problems. It's often preferred when the graph is dense or easily accessible from a single starting point. Example: Growing a city's power grid by always connecting the nearest new city to the existing grid.

```
import heapq
def prim(graph):
    # graph: dict node -> list of (cost, node)
    start = next(iter(graph))
    visited = {start}
    edges = graph[start][:]
    heapq.heapify(edges)
    mst = []
    while edges:
        cost, u, v = heapq.heappop(edges)
        if v not in visited:
            visited.add(v)
            mst.append((u, v, cost))
            for edge in graph[v]:
                heapq.heappush(edges, edge)
    return mst

# Example usage
graph = {
```

```
    0: [(5,0,2),(7,0,1)],
    1: [(7,1,0),(8,1,2),(9,1,3)],
    2: [(5,2,0),(8,2,1),(7,2,3),(5,2,4)],
    3: [(9,3,1),(7,3,2),(15,3,4),(6,3,5)],
    4: [(5,4,2),(15,4,3),(8,4,5),(9,4,6)],
    5: [(6,5,3),(8,5,4),(11,5,6)],
    6: [(9,6,4),(11,6,5)]
}
mst = prim(graph)
print(mst)  # Example MST edges
```

Explanation: Pick an arbitrary `start` node, mark it visited, and push all its edges onto a heap. Repeatedly pop the cheapest edge connecting to an unvisited vertex, add that edge to `mst`, mark the new vertex visited, and push its edges.
Analogy: *"Expanding a network like adding nearest neighboring cities to an existing connected cluster, one by one by cheapest route."*

Dijkstra's (Shortest Paths)

[14] [15] Dijkstra's Algorithm finds the shortest paths from one source node to all other nodes in a graph with **non-negative weights**. It initializes distances to infinity (0 at the source), then repeatedly picks the unvisited node with the smallest known distance (using a min-priority queue) and relaxes its neighbors (updates their distances if a shorter path is found via this node).

- **Time Complexity:** $O((E+V)\log V)$ with a binary heap (or $O(E+V\log V)$); **Space:** $O(V)$.
- **When/Why Used:** For routing and shortest-path problems with non-negative edge weights. Example: finding shortest driving distances from one city to all others on a map. It's the core of many GPS and network protocols.

```
import heapq
def dijkstra(graph, src):
    # graph: dict node -> list of (neighbor, weight)
    dist = {node: float('inf') for node in graph}
    dist[src] = 0
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))
    return dist

# Example usage
```

```
graph = {
    'A': [('B',5), ('C',1)],
    'B': [('C',2), ('D',1)],
    'C': [('B',2), ('D',4), ('E',8)],
    'D': [('E',3)],
    'E': []
}
print(dijkstra(graph, 'A'))  # Shortest distances from 'A' to all others
```

Explanation: Maintain a `dist` map and a priority queue of `(distance, node)`. Pop the smallest distance node, and for each neighbor, update (`relax`) `dist[neighbor]` if going through this node is shorter. Continue until all nodes are processed.
Analogy: *"Like mapping out the shortest driving distance to every city from your starting point, picking the closest next city each time."*

Depth-First Search (DFS)

16 17 Depth-First Search explores as far along each branch as possible before backtracking. Starting from a source, it visits a node, then recursively explores an unvisited neighbor until reaching a dead end, then backtracks. DFS uses a stack (often via recursion) to remember nodes to return to.

- **Time Complexity:** $O(V + E)$; **Space:** $O(V)$ (recursion stack or visited list).
- **When/Why Used:** Traversing or searching graph/tree structures, checking connectivity, finding paths, topological sorting, etc. Example: exploring a maze by going down one path completely before trying alternatives.

```
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    for neighbor in graph.get(start, []):
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
    return visited

# Example usage
g = {'A':['B','C'], 'B':['D'], 'C':['E'], 'D':[], 'E':[]}
print(dfs(g, 'A'))  # Output: {'A', 'B', 'C', 'D', 'E'}
```

Explanation: Mark `start` as visited, then recursively `dfs` on each unvisited neighbor. This visits all nodes reachable from `start`.
Analogy: *"Like exploring a cave by always going forward until you hit a wall, then backtracking and choosing a new path."*

Breadth-First Search (BFS)

[18] [19] Breadth-First Search explores a graph level by level. Starting from a source node, it first visits all neighbors (distance 1), then all neighbors of neighbors (distance 2), and so on. It uses a queue to process nodes in the order discovered.

- **Time Complexity:** $O(V + E)$; **Space:** $O(V)$.
- **When/Why Used:** Finding the shortest path in unweighted graphs, level-order traversal of trees, or any scenario where you need the minimum number of steps. Example: finding the shortest route in a city grid (assuming each road has equal length).

```
from collections import deque
def bfs(graph, start):
    visited = {start}
    queue = deque([start])
    order = []
    while queue:
        v = queue.popleft()
        order.append(v)
        for neighbor in graph.get(v, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order

# Example usage
g = {'A':['B','C'], 'B':['D'], 'C':['E'], 'D':[], 'E':[]}
print(bfs(g, 'A'))  # Output: ['A', 'B', 'C', 'D', 'E']
```

Explanation: We start with `start` in a queue. Repeatedly dequeue the front node, record it, and enqueue all its unvisited neighbors. This visits nodes in increasing distance from the start.
Analogy: *"Like ripples in water: exploring all immediate neighbors first, then their neighbors, expanding outward evenly."*

Bellman-Ford

[20] [21] Bellman-Ford is a shortest-path algorithm that can handle **negative edge weights** (but no negative cycles). It relaxes all edges repeatedly: in each of $V - 1$ iterations (where V is vertex count) it tries to improve distances along every edge. If a shorter path is found, it updates distances. After $V - 1$ passes, all shortest distances (if no negative cycles) are found.

- **Time Complexity:** $O(V \times E)$; **Space:** $O(V)$.
- **When/Why Used:** When graphs may have negative edges (unlike Dijkstra). Example: computing shortest paths in currency exchange graphs to detect arbitrage (negative cycle detection) or any problem with negative costs.

```python
def bellman_ford(edges, num_vertices, source):
    dist = [float('inf')] * num_vertices
    dist[source] = 0
    for _ in range(num_vertices - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    # (Optional) Check for negative cycles
    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            raise ValueError("Negative-weight cycle detected")
    return dist

# Example usage (edges = (u, v, weight), 5 vertices)
edges = [
    (0, 1, 6), (0, 2, 7),
    (1, 3, 5), (1, 2, 8), (1, 4, -4),
    (2, 3, -3), (2, 4, 9),
    (3, 1, -2),
    (4, 3, 7), (4, 0, 2)
]
print(bellman_ford(edges, 5, 0))  # Shortest distances from vertex 0
```

Explanation: Initialize `dist[source] = 0`, others infinity. Relax (update) all edges $V - 1$ times. After this, `dist[v]` holds the shortest path length. A final pass checks for negative cycles.
Analogy: *"Like waves of updates: each round spreads improved distances, until even the farthest nodes settle on their shortest paths."*

Floyd–Warshall (All-Pairs Shortest Paths)

22 23 Floyd–Warshall computes shortest paths **between all pairs of vertices** in a weighted graph (allowing positive or negative weights, but no negative cycles). It starts with a distance matrix (direct edge weights or infinity), then iteratively updates: for each intermediate vertex k, it sets `dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])`. In the end, `dist[i][j]` is the shortest distance from i to j.

- **Time Complexity:** $O(V^3)$; **Space:** $O(V^2)$ (for the distance matrix).
- **When/Why Used:** When you need all-pairs shortest paths and the graph is not too large. Example: computing quickest routes between every pair of cities, or finding transitive closure of routes.

```python
def floyd_warshall(n, dist):
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
```

```
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist

# Example usage (4 vertices, initial distance matrix)
inf = float('inf')
dist = [
    [0,   5,   inf, 10],
    [inf, 0,   3,   inf],
    [inf, inf, 0,   1],
    [inf, inf, inf, 0]
]
print(floyd_warshall(4, dist))
```

Explanation: We treat `dist[i][j]` as the current best. For each `k`, we see if going from `i` to `j` via `k` is shorter, and update accordingly. This triple loop gradually accounts for all possible intermediate vertices. **Analogy:** *"Like checking if the quickest path from city A to B is faster by going through city C, and doing this for all intermediate cities one by one."*

Topological Sort (DAG Ordering)

[24] [25] A topological sort of a directed acyclic graph (DAG) is a linear ordering of vertices such that for every directed edge $u \to v$, u comes before v. It can be done via DFS (post-order reverse) or Kahn's algorithm (BFS). Essentially, it schedules tasks given dependency constraints.

- **Time Complexity:** $O(V+E)$; **Space:** $O(V+E)$.
- **When/Why Used:** Any scenario with dependencies (build systems, course prerequisite planning, task scheduling). It orders tasks so that each task's prerequisites appear earlier. Example: determining the order to compile files where some files depend on others.

```
def topo_sort(graph):
    visited = set()
    stack = []
    def dfs(u):
        visited.add(u)
        for v in graph.get(u, []):
            if v not in visited:
                dfs(v)
        stack.append(u)
    for node in graph:
        if node not in visited:
            dfs(node)
    return stack[::-1]  # reverse post-order

# Example usage (DAG)
g = {
    'A': ['C'], 'B': ['C','D'], 'C': ['E'], 'D': ['F'],
```

```
    'E': ['H','F'], 'F': [], 'G': ['H'], 'H': []
}
print(topo_sort(g))  # One possible topological order
```

Explanation: The DFS-based method visits all nodes, adding each node to a stack only after its descendants. Reversing that list yields a valid topological order.
Analogy: *"Like arranging tasks in a to-do list so that all prerequisites come before the task."*

0/1 Knapsack (DP)

[26] [27] The 0/1 Knapsack Problem is an optimization problem: given items with weights and values and a knapsack capacity W, choose a subset to maximize value without exceeding capacity. The **dynamic programming** solution uses a 2D table `dp[i][w]` = max value using first i items with weight limit w. We build up this table iteratively.

- **Time Complexity:** $O(nW)$ (pseudo-polynomial, where n is number of items and W capacity); **Space:** $O(nW)$.
- **When/Why Used:** Resource allocation, budgeting, selection problems. Example: choosing which projects to fund given budget constraints, or which combination of goods to pack given weight limit.

```
def knapsack_01(weights, values, W):
    n = len(values)
    dp = [[0]*(W+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(W+1):
            dp[i][w] = dp[i-1][w]
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w], dp[i-1][w-weights[i-1]] + values[i-1])
    return dp[n][W]

# Example usage
weights = [1, 2, 3]
values = [6, 10, 12]
W = 5
print(knapsack_01(weights, values, W))  # Output: 22
```

Explanation: We fill `dp[i][w]` by either excluding item i (taking `dp[i-1][w]`) or including it (`values[i-1] + dp[i-1][w-weights[i-1]]`) if it fits. The answer is `dp[n][W]`.
Analogy: *"Filling a backpack with items: you try combinations of items up to the weight limit to get the most value."*

Coin Change (DP)

[28] [29] The Coin Change problem asks for the minimum number of coins to make a target amount. A standard DP solution computes `dp[x]` = min coins to make amount `x`, building up from 0 to the target. For each coin, you update the dp table so `dp[x] = min(dp[x], dp[x-coin] + 1)`.

- **Time Complexity:** $O(nW)$ where n is number of coin types and W is the target amount; **Space:** $O(W)$.
- **When/Why Used:** Currency systems, making change, or any problem of partitioning an amount with given denominations. Example: find the fewest coins (quarters, dimes, etc.) to make \$1.00.

```
def coin_change(coins, amount):
    dp = [float('inf')] * (amount+1)
    dp[0] = 0
    for coin in coins:
        for x in range(coin, amount+1):
            dp[x] = min(dp[x], dp[x-coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1

# Example usage
coins = [1, 2, 5]
print(coin_change(coins, 11))  # Output: 3 (5+5+1)
```

Explanation: `dp[x]` tracks the fewest coins to make `x`. We iterate coins, and for each coin update possible amounts. Finally `dp[amount]` is the answer.
Analogy: *"Like making change for a dollar: try larger coins first, but DP systematically tries all combinations to minimize coins."*

Longest Common Subsequence (LCS)

[30] [31] LCS finds the longest subsequence common to two sequences (not necessarily contiguous). Dynamic programming builds a 2D table `dp[i][j]` = length of LCS of the first `i` characters of `X` and first `j` of `Y`. If `X[i-1] == Y[j-1]`, then `dp[i][j] = dp[i-1][j-1] + 1`; otherwise `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`.

- **Time Complexity:** $O(n \times m)$ for strings of lengths n and m; **Space:** $O(nm)$.
- **When/Why Used:** Comparing sequences: diff tools, bioinformatics (DNA sequence alignment), version control. Example: finding the longest matching line sequence between two documents.

```
def lcs(X, Y):
    m, n = len(X), len(Y)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(m):
        for j in range(n):
```

```
            if X[i] == Y[j]:
                dp[i+1][j+1] = dp[i][j] + 1
            else:
                dp[i+1][j+1] = max(dp[i][j+1], dp[i+1][j])
    return dp[m][n]

# Example usage
print(lcs("AGGTAB", "GXTXAYB"))  # Output: 4 (LCS is "GTAB")
```

Explanation: Build a matrix where each cell is LCS length for prefixes. If characters match, take diagonal +1; else take the max of up or left. The final cell has the answer.
Analogy: *"Like finding the longest tale that both stories share, by comparing their plots step-by-step."*

Longest Increasing Subsequence (LIS)

32 33 LIS finds the longest strictly increasing subsequence in an array. A classic $O(n \log n)$ solution maintains a list where `sub[k]` is the smallest tail of all increasing subsequences of length `k+1`. For each element, use binary search to place it in `sub`.

- **Time Complexity:** $O(n \log n)$; **Space:** $O(n)$.
- **When/Why Used:** Pattern analysis, version control (finding differences), or longest rising trends. Example: longest sequence of steadily rising stock prices.

```
def lis(seq):
    from bisect import bisect_left
    sub = []
    for x in seq:
        i = bisect_left(sub, x)
        if i == len(sub):
            sub.append(x)
        else:
            sub[i] = x
    return len(sub)

# Example usage
print(lis([10, 9, 2, 5, 3, 7, 101, 18]))  # Output: 4 (e.g. [2,3,7,101])
```

Explanation: We maintain a sorted list `sub` of candidate ends. For each `x`, find where it fits to possibly extend an existing sequence or start a new longer one. The length of `sub` is the LIS length.
Analogy: *"Picking cards and always placing each card on the first pile whose top is ≥ the card; number of piles = LIS length."*

Fibonacci (DP)

Computing Fibonacci numbers efficiently is a classic DP example. The naive recursive fib grows exponentially, but a DP approach builds up either via memoization or bottom-up. For instance, iteratively compute `fib(i) = fib(i-1) + fib(i-2)` storing results.

- **Time Complexity:** $O(n)$ with DP; naive recursion is $O(2^n)$. **Space:** $O(n)$ or $O(1)$ if optimized.
- **When/Why Used:** Generating Fibonacci numbers quickly, or in similar recurrence problems. Example: finding the nth Fibonacci number or counting ways to climb stairs.

```
def fib(n):
    if n <= 1:
        return n
    dp = [0, 1]
    for i in range(2, n+1):
        dp.append(dp[i-1] + dp[i-2])
    return dp[n]

# Example usage: first 10 Fibonacci numbers
print([fib(i) for i in range(10)])  # [0,1,1,2,3,5,8,13,21,34]
```

Explanation: We build a list `dp` of Fibonacci numbers from 0 up to n, using the recurrence $F[i] = F[i-1] + F[i-2]$.
Analogy: *"Like finding how many ways to climb n stairs by summing the ways to reach the previous two steps."*

Matrix Chain Multiplication (DP)

34 Matrix Chain Multiplication chooses the optimal order to multiply a chain of matrices. Given dimensions $d_0 \times d_1, d_1 \times d_2, \ldots$, the cost of multiplying $A_i \times A_{i+1}$ depends on dimensions. A DP approach defines `dp[i][j]` = minimum cost (scalar multiplications) to multiply matrices from i to j. Then

$$\mathrm{dp}[i][j] = \min_{k=i..j-1} (\mathrm{dp}[i][k] + \mathrm{dp}[k+1][j] + d_{i-1} * d_k * d_j).$$

- **Time Complexity:** $O(n^3)$ (for n matrices); **Space:** $O(n^2)$.
- **When/Why Used:** In computer graphics, compiler optimization, or any context where multiple matrix multiplications occur. Example: Determining the most efficient order to perform a series of matrix multiplications in a calculation.

```
from functools import lru_cache
def matrix_chain_order(dims):
    @lru_cache(None)
    def cost(i, j):
        if j - i <= 1:
            return 0
```

```
        return min(cost(i, k) + cost(k, j) + dims[i]*dims[k]*dims[j]
                   for k in range(i+1, j))
    return cost(0, len(dims)-1)

# Example usage: matrices A(10x30), B(30x5), C(5x60)
dims = [10, 30, 5, 60]
print(matrix_chain_order(tuple(dims)))  # Output: 4500
```

Explanation: We use a memoized recursive function `cost(i,j)` that tries all split points `k` to find the best. `dims` stores matrix dimensions; the multiplication cost is $dims[i] \times dims[k] \times dims[j]$.
Analogy: *"Parenthesizing a sequence of matrix multiplications: like planning which parentheses to put in a long product to minimize work."*

Tree Traversals (In/Pre/Post)

35 Tree traversal algorithms visit all nodes of a tree in a particular order. Common depth-first orders on a binary tree are: Pre-order (Node, Left, Right), In-order (Left, Node, Right), and Post-order (Left, Right, Node). These recursively visit subtrees. Traversal algorithms typically run in $O(n)$ time, visiting each of the n nodes once.

- **Time Complexity:** $O(n)$ for n nodes; **Space:** $O(h)$ where h is tree height (due to recursion or a stack).
- **When/Why Used:** To inspect or process all nodes in a tree. Pre-order is used to copy trees, in-order retrieves sorted order from BST, post-order for deleting trees. Example: printing all values in a binary tree in sorted order (in-order traversal on BST).

```
class Node:
    def __init__(self, key, left=None, right=None):
        self.key = key
        self.left = left
        self.right = right

def inorder(root):
    return inorder(root.left) + [root.key] + inorder(root.right) if root else []

# Example usage: build tree and traverse in-order
root = Node('A', Node('B', Node('D'), Node('E')), Node('C', None, Node('F')))
print(inorder(root))  # Output: ['D', 'B', 'E', 'A', 'C', 'F']
```

Explanation: The example shows in-order (left-root-right). The function uses recursion: it returns the list from left subtree, then root key, then right subtree. Pre-order or post-order would differ in where the root is appended.
Analogy: *"Like reading a family tree: pre-order (visit parent, then children), in-order (left child, parent, right child), post-order (children, then parent)."*

Binary Search Tree (BST) Operations

[36] [37] A Binary Search Tree is a binary tree where each node's key is greater than all keys in its left subtree and less than all in its right subtree. Operations like search, insert, and delete run in $O(h)$ time where h is the height. On average $h = O(\log n)$, worst-case $h = O(n)$ (unbalanced).

- **Time Complexity:** Average $O(\log n)$ for search/insert/delete; Worst-case $O(n)$ (if the tree is a chain); **Space:** $O(n)$.
- **When/Why Used:** When you need a sorted data structure with fast insert and lookup. Example: maintaining a sorted set of numbers where you frequently add or search elements.

```
class BST:
    def __init__(self, val):
        self.val = val
        self.left = self.right = None

    def insert(self, x):
        if x < self.val:
            if self.left: self.left.insert(x)
            else: self.left = BST(x)
        else:
            if self.right: self.right.insert(x)
            else: self.right = BST(x)

    def search(self, x):
        if x == self.val: return True
        if x < self.val:
            return self.left.search(x) if self.left else False
        else:
            return self.right.search(x) if self.right else False

# Example usage
root = BST(10)
for num in [5, 15, 3, 7]:
    root.insert(num)
print(root.search(7))   # True
print(root.search(12))  # False
```

Explanation: `insert(x)` follows left/right pointers until finding the right place for `x`. `search(x)` compares `x` with the current node and recurses left or right.
Analogy: *"Like a family tree sorted by last names: to find a person, you go left/right depending on alphabetical order."*

Segment Tree (Range Queries)

[38] [39] A Segment Tree is a binary-tree data structure for answering range queries (like sum or min on subarrays) in $O(\log n)$ time, with $O(n)$ space. Leaf nodes represent individual array elements, and internal nodes represent an aggregate (e.g., sum) of a segment. Building it takes $O(n \log n)$ or $O(n)$ (bottom-up), and queries/updates take $O(\log n)$.

- **Time Complexity:** Build in $O(n)$ or $O(n \log n)$, Query/Update in $O(\log n)$; **Space:** $O(n)$.
- **When/Why Used:** For dynamic array queries and updates on ranges. Example: computing the sum of elements in a subarray that can change over time (frequent updates and queries).

```
class SegmentTree:
    def __init__(self, data):
        self.n = len(data)
        self.tree = [0] * (2*self.n)
        # build tree by copying leaves then computing parents
        for i in range(self.n):
            self.tree[self.n+i] = data[i]
        for i in range(self.n-1, 0, -1):
            self.tree[i] = self.tree[2*i] + self.tree[2*i+1]
    def query(self, l, r):
        # query sum on interval [l, r)
        res = 0
        l += self.n; r += self.n
        while l < r:
            if l & 1:
                res += self.tree[l]; l += 1
            if r & 1:
                r -= 1; res += self.tree[r]
            l //= 2; r //= 2
        return res

# Example usage
arr = [1,2,3,4,5]
seg = SegmentTree(arr)
print(seg.query(1,4))  # sum of arr[1:4] = 2+3+4 = 9
```

Explanation: We build a tree of size $2n$. The second half stores the original array; the first half stores sums of segments. The `query(l,r)` method moves two pointers up the tree, adding segments fully covered. **Analogy:** *"Like a tournament bracket: each node holds the sum of a group; you can combine segments by climbing up and adding partial sums."*

Fenwick Tree (Binary Indexed Tree)

[40] [41] A Fenwick Tree (Binary Indexed Tree) efficiently computes prefix sums and updates on an array in $O(\log n)$ time. It stores cumulative information in a clever implicit tree inside an array. Each index `i` covers a range of values ending at `i` determined by its least significant bit.

- **Time Complexity:** Update and query both $O(\log n)$; **Space:** $O(n)$.
- **When/Why Used:** When you need fast prefix-sum queries and point updates. Example: dynamically computing cumulative frequencies (like maintaining prefix sums of votes or scores as they change).

```
class Fenwick:
    def __init__(self, n):
        self.n = n
        self.bit = [0]*(n+1)
    def update(self, i, delta):
        while i <= self.n:
            self.bit[i] += delta
            i += i & -i
    def query(self, i):
        s = 0
        while i > 0:
            s += self.bit[i]
            i -= i & -i
        return s

# Example usage
fenw = Fenwick(5)
data = [1,2,3,4,5]
for i, val in enumerate(data,1):
    fenw.update(i, val)
print(fenw.query(3))  # prefix sum up to index 3: 1+2+3 = 6
```

Explanation: We treat `bit` as 1-indexed. `update(i,delta)` adds `delta` to index `i` and all relevant ancestors (using `i & -i`). `query(i)` sums up appropriate elements to get the prefix sum.
Analogy: *"Each element holds a partial sum of the last 2^k elements; climbing up through bits lets you quickly get the total."*

Knuth–Morris–Pratt (KMP)

[42] [43] KMP is a string-search algorithm that finds occurrences of a pattern in a text in $O(n+m)$ time (n, m = lengths) by using the pattern itself to skip comparisons. It preprocesses the pattern to build an "lps" (longest proper prefix which is also suffix) table, then scans the text. On mismatch, it uses the table to know how far to skip in the pattern without re-checking matched characters.

- **Time Complexity:** $O(n+m)$ where n = length of text, m = length of pattern; **Space:** $O(m)$.

- **When/Why Used:** Fast substring search when worst-case linear time is needed. Example: finding a word in a long document.

```python
def kmp_search(text, pattern):
    # Build LPS (longest prefix suffix) table
    n, m = len(text), len(pattern)
    lps = [0]*m
    length = 0; i = 1
    while i < m:
        if pattern[i] == pattern[length]:
            length += 1; lps[i] = length; i += 1
        elif length:
            length = lps[length-1]
        else:
            i += 1
    # Search using LPS
    res = []
    i = j = 0
    while i < n:
        if text[i] == pattern[j]:
            i += 1; j += 1
            if j == m:
                res.append(i - j)
                j = lps[j-1]
        else:
            if j:
                j = lps[j-1]
            else:
                i += 1
    return res

# Example usage
text = "ababcabcababc"
pattern = "abc"
print(kmp_search(text, pattern))  # Output: [2, 5, 9]
```

Explanation: We first fill `lps` so that `lps[i]` tells the longest prefix ending at `i`. During search, when a mismatch happens, we use `lps[j-1]` to shift the pattern. This avoids rescanning.
Analogy: *"Like finding a pattern in text by remembering the last matched part so you don't go back all the way after a mismatch."*

Rabin–Karp (String Search)

[44] [45] Rabin–Karp searches for a pattern in text using a rolling hash. It hashes the pattern and then slides a window of text computing its hash efficiently. Whenever the hash matches, it does a character-by-character check. Expected time is $O(n+m)$, though worst-case is $O(nm)$ (due to hash collisions).

- **Time Complexity:** Average $O(n+m)$; Worst-case $O(nm)$; **Space:** $O(1)$ besides storing hashes.
- **When/Why Used:** For multiple-pattern search or plagiarism detection (comparing large texts). Example: checking if a sentence appears in a large document.

```
def rabin_karp(text, pattern):
    n, m = len(text), len(pattern)
    if m > n: return []
    base, mod = 256, 101  # example base and a prime modulus
    hpattern = 0; htext = 0; h = pow(base, m-1, mod)
    # Initial hash for pattern and first window
    for i in range(m):
        hpattern = (base*hpattern + ord(pattern[i])) % mod
        htext = (base*htext + ord(text[i])) % mod
    res = []
    # Slide over text
    for i in range(n-m+1):
        if hpattern == htext and text[i:i+m] == pattern:
            res.append(i)
        if i < n-m:
            htext = (htext - ord(text[i])*h) % mod
            htext = (htext * base + ord(text[i+m])) % mod
            htext = (htext + mod) % mod
    return res

# Example usage
print(rabin_karp("abracadabra", "abra"))  # Output: [0, 7]
```

Explanation: We compute rolling hash of each substring of length `m` in `text`, comparing to the hash of `pattern`. If hashes match, we verify the substring. Rolling hash update shifts the window by removing the leading char and adding a new one.
Analogy: *"Like converting text chunks into numeric fingerprints and quickly tossing out non-matches."*

Trie (Prefix Tree)

[46] A Trie (prefix tree) is a tree structure that stores a set of strings (or keys) where each node corresponds to a prefix. In a Trie, common prefixes share nodes, enabling efficient lookup. Each edge is labeled with a

character, and a boolean flag indicates if a node terminates a valid key. Typical operations (insert, search) take $O(m)$ time for strings of length m.

- **Time Complexity:** Search/Insert/Delete $O(m)$ (m = length of key); **Space:** $O(n \cdot w)$ where n keys of average length w (due to many pointers).
- **When/Why Used:** Autocomplete, spell-check, IP routing (prefix matching), storing dictionaries. Example: checking if a word exists in a dictionary or retrieving all words with a given prefix.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()
    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.end = True
    def search(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return node.end

# Example usage
trie = Trie()
trie.insert("hello")
trie.insert("world")
print(trie.search("hello"))  # True
print(trie.search("worlds")) # False
```

Explanation: `insert` follows each character, creating nodes as needed. `search` follows nodes for each character; if it reaches the end flag, the word exists.
Analogy: *"A phonebook tree where each level is a letter: to find a word, you follow its letters down the branches."*

Two Pointers (Technique)

Two-pointers is a common technique using two indices (often one at the start and one at the end, or both moving through the array) to solve problems in linear time. It's not a single algorithm but a pattern. It's

frequently used on **sorted arrays or linked lists** to find pairs, remove duplicates, or partition data. For example, finding if two numbers sum to a target in a sorted array: move one pointer forward or backward based on the sum.

- **Time Complexity:** Typically $O(n)$; **Space:** $O(1)$.
- **When/Why Used:** When the data is sorted or when you need to compare elements from two ends or sliding windows. Examples: two-sum in sorted array, partitioning, merging two sorted lists.

```
def two_sum_sorted(arr, target):
    i, j = 0, len(arr) - 1
    while i < j:
        current = arr[i] + arr[j]
        if current == target:
            return (i, j)
        elif current < target:
            i += 1
        else:
            j -= 1
    return None

# Example usage (array must be sorted)
print(two_sum_sorted([1,2,3,4,4,9], 8))  # Output: (3, 4) since arr[3]+arr[4] =
4+4 = 8
```

Explanation: We start with `i` at the left and `j` at the right. If the sum is too small, we move `i` right; if too large, move `j` left. This finds a pair adding to `target` in one pass.
Analogy: *"Like two people starting at both ends of a line of numbers and walking inward to find the pair that adds up correctly."*

Z-Algorithm (Pattern Matching)

[47] The Z-algorithm preprocesses a string `S` of length `n` by computing an array `Z` where `Z[i]` is the length of the longest substring starting at `i` that is also a prefix of `S`. It runs in $O(n)$ time. This can be used for pattern matching by concatenating `pattern + '$' + text` and computing Z; whenever a `Z[i]` equals the pattern's length, a match is found.

- **Time Complexity:** $O(n)$ to compute the Z-array for a string of length n; **Space:** $O(n)$.
- **When/Why Used:** String matching (similar to KMP), finding patterns or periodicities in strings. Example: searching for a pattern in text by building `T = pattern + '$' + text`.

```
def compute_z_array(s):
    n = len(s)
    Z = [0]*n
    l = r = 0
```

```
    for i in range(1, n):
        if i <= r:
            Z[i] = min(r - i + 1, Z[i - l])
        while i + Z[i] < n and s[Z[i]] == s[i + Z[i]]:
            Z[i] += 1
        if i + Z[i] - 1 > r:
            l, r = i, i + Z[i] - 1
    return Z

# Example usage
text = "abaacaba"
print(compute_z_array(text))  # Z array values
```

Explanation: We maintain a window $[l, r]$ where the substring matches the prefix. For each `i`, we initialize `Z[i]` based on previous values, then extend it by character comparisons.
Analogy: *"For each position, the Z-array tells how far you can match the prefix; it's like measuring prefix-match lengths at every point."*

---

1 Linear search - Wikipedia
https://en.wikipedia.org/wiki/Linear_search

2 3 Binary search - Wikipedia
https://en.wikipedia.org/wiki/Binary_search

4 Merge sort - Wikipedia
https://en.wikipedia.org/wiki/Merge_sort

5 6 Quicksort - Wikipedia
https://en.wikipedia.org/wiki/Quicksort

7 8 Heapsort - Wikipedia
https://en.wikipedia.org/wiki/Heapsort

9 Activity selection problem - Wikipedia
https://en.wikipedia.org/wiki/Activity_selection_problem

10 Huffman coding - Wikipedia
https://en.wikipedia.org/wiki/Huffman_coding

11 12 Kruskal's algorithm - Wikipedia
https://en.wikipedia.org/wiki/Kruskal%27s_algorithm

13 Prim's algorithm - Wikipedia
https://en.wikipedia.org/wiki/Prim%27s_algorithm

14 15 Dijkstra's algorithm - Wikipedia
https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm

16 17 Depth-first search - Wikipedia
https://en.wikipedia.org/wiki/Depth-first_search

18 19 Breadth-first search - Wikipedia
https://en.wikipedia.org/wiki/Breadth-first_search

20 21 Bellman–Ford algorithm - Wikipedia
https://en.wikipedia.org/wiki/Bellman%E2%80%93Ford_algorithm

22 Floyd–Warshall algorithm - Wikipedia
https://en.wikipedia.org/wiki/Floyd%E2%80%93Warshall_algorithm

23 Time and Space Complexity of Floyd Warshall Algorithm - GeeksforGeeks
https://www.geeksforgeeks.org/dsa/time-and-space-complexity-of-floyd-warshall-algorithm/

24 25 Topological sorting - Wikipedia
https://en.wikipedia.org/wiki/Topological_sorting

26 27 Knapsack problem - Wikipedia
https://en.wikipedia.org/wiki/Knapsack_problem

28 29 Change-making problem - Wikipedia
https://en.wikipedia.org/wiki/Change-making_problem

30 31 Longest common subsequence - Wikipedia
https://en.wikipedia.org/wiki/Longest_common_subsequence

32 33 Longest increasing subsequence - Wikipedia
https://en.wikipedia.org/wiki/Longest_increasing_subsequence

34 Matrix chain multiplication - Wikipedia
https://en.wikipedia.org/wiki/Matrix_chain_multiplication

35 Tree traversal - Wikipedia
https://en.wikipedia.org/wiki/Tree_traversal

36 37 Binary search tree - Wikipedia
https://en.wikipedia.org/wiki/Binary_search_tree

38 39 Segment tree - Wikipedia
https://en.wikipedia.org/wiki/Segment_tree

40 41 Fenwick tree - Wikipedia
https://en.wikipedia.org/wiki/Fenwick_tree

42 43 Knuth–Morris–Pratt algorithm - Wikipedia
https://en.wikipedia.org/wiki/Knuth%E2%80%93Morris%E2%80%93Pratt_algorithm

44 45 Rabin–Karp algorithm - Wikipedia
https://en.wikipedia.org/wiki/Rabin%E2%80%93Karp_algorithm

46 Trie - Wikipedia
https://en.wikipedia.org/wiki/Trie

47 Z Algorithm - Codeforces
https://codeforces.com/blog/entry/3107